Lecture 29: MIPS Procedures (Functions) & Stack Basics

🎯 Learning Objectives

By the end of this lecture, students will be able to:

- Understand why procedures (functions) are used.
- Explain the MIPS calling convention.
- Use `jal` and `jr` correctly.
- Pass parameters using `$a0–$a3`.
- Return values using `$v0`.
- Understand the purpose of the stack.
- Save and restore registers using the stack.
- Write reusable MIPS procedures.

---

1. Introduction

🎤 Teaching Script

First 30-Second SEO Phrase

Naturally mention:

> **"In today's Computer Organization and Assembly Language lecture, we'll learn how to create reusable procedures in MIPS Assembly Language, understand the MIPS calling convention, use `jal` and `jr`, and explore the basics of the stack with practical examples in the MARS simulator."**

In our previous lecture, we learned how to work with strings in MIPS Assembly Language.

Today, we're going to learn one of the most important concepts in programming: procedures, also known as functions.

Imagine you're writing a program that needs to calculate the sum of two numbers in ten different places. Instead of writing the same code ten times, you can write it once inside a procedure and call it whenever you need it.

This makes programs shorter, easier to understand, and easier to maintain.

In MIPS, procedures are implemented using instructions such as `jal` and `jr`, along with a special memory area called the stack.

By the end of this lecture, you'll be able to write your own reusable procedures and understand how the processor manages function calls.

Let's begin.

---

2. What is a Procedure?

A **procedure** (or function) is a reusable block of code that performs a specific task.

Example in C++:

```
int add(int a, int b)

{

   return a + b;

}


int main()

{

   int result = add(10, 20);
```

}

Instead of repeating code, we **call** the function.

---

3. Procedures in MIPS

A procedure consists of:

- A **label**
- A sequence of instructions
- A return instruction (`jr $ra`)

Example:

```
main:

    jal Sum

    li $v0,10

    syscall

Sum:

    # Procedure Body

    jr $ra
```

---

4. Understanding `jal`

`jal` stands for:

Jump And Link

When executed:

1. Saves the return address in `$ra`.
2. Jumps to the procedure.

Example:

jal Sum

---

5. Understanding `jr`

`jr` stands for:

Jump Register

Usually:

jr $ra

This returns execution to the instruction immediately after the `jal`.

---

6. Program Flow

main

↓

jal Sum

↓

Sum Procedure

↓

jr $ra

↓

Back to main

This is the basic mechanism behind every function call in MIPS.

---

7. MIPS Calling Convention

MIPS defines specific registers for passing data between procedures.

Register	Purpose
`$a0–$a3`	Function arguments
`$v0–$v1`	Return values
`$ra`	Return address
`$sp`	Stack pointer
`$t0–$t9`	Temporary registers (caller-saved)
`$s0–$s7`	Saved registers (callee-saved)

Explain that following this convention allows different procedures to work together correctly.

---

8. Passing Parameters

Suppose:

add(10,20)

MIPS:

li $a0,10

li $a1,20

jal Add

The first argument goes into `$a0`, the second into `$a1`.

---

9. Returning a Value

The return value is placed in `$v0`.

Example:

Add:

```
add $v0,$a0,$a1

jr $ra
```

Back in `main`:

move $t0,$v0

Now:

$t0 = 30

---

10. Complete Procedure Example

```
.text
.globl main

main:

    li $a0,15
    li $a1,25

    jal Add

    move $t0,$v0

    li $v0,10
    syscall

Add:

    add $v0,$a0,$a1
```

```
    jr $ra
```

After execution:

$t0 = 40

---

11. Why Do We Need the Stack?

Imagine one procedure calls another.

Example:

main

↓

Function A

↓

Function B

↓

Function C

Each function needs to remember:

- Where to return
- Important register values

The **stack** stores this information.

---

12. What is the Stack?

The **stack** is a special memory area that follows the **Last In, First Out (LIFO)** principle.

Example:

Push A

Push B

Push C

Pop C

Pop B

Pop A

The last item placed on the stack is the first one removed.

---

13. Stack Pointer ($sp)

`$sp` points to the **top of the stack**.

To allocate space:

addi $sp,$sp,-4

To free space:

addi $sp,$sp,4

---

14. Saving the Return Address

Before calling another procedure inside a procedure, save `$ra`.

addi $sp,$sp,-4

sw $ra,0($sp)

Restore it before returning:

lw $ra,0($sp)

addi $sp,$sp,4

This ensures the correct return address is preserved.

---

15. Saving Registers

If a procedure modifies a saved register (`$s0–$s7`), it should save and restore it.

Example:

addi $sp,$sp,-4

sw $s0,0($sp)

...

lw $s0,0($sp)

addi $sp,$sp,4

Explain the difference between **caller-saved** (`$t`) and **callee-saved** (`$s`) registers.

---

16. Visualizing the Stack

Higher Memory

———————————

Previous Data

Saved $ra

Saved $s0

← $sp

Lower Memory

Remember:

The stack **grows downward** (toward lower memory addresses).

---

17. Procedure with Parameters

Example:

```
int square(int x)
{
    return x * x;
```

}

MIPS:

```
main:

    li $a0,6

    jal Square

    move $t0,$v0

    li $v0,10
    syscall

Square:

    mul $v0,$a0,$a0

    jr $ra
```

Result:

$t0 = 36

---

18. Procedure Calling Another Procedure

Example:

main

↓

Add

↓

Multiply

↓

Return

↓

Return

Explain that each nested call must preserve the return address, often using the stack.

---

19. Real-Life Analogy

Imagine a receptionist.

- A customer (caller) requests a service.
- The receptionist passes the request to a specialist (procedure).
- The specialist completes the work and returns the result.
- The receptionist continues with the next customer.

The stack acts like a file where ongoing tasks are temporarily stored until they are completed.

---

20. Common Mistakes

Students often:

❌ Forget to return.

Every procedure should end with:

jr $ra

❌ Forget to store the return value in `$v0`.

Incorrect:

add $t0,$a0,$a1

Correct:

add $v0,$a0,$a1

❌ Overwrite `$ra` in nested procedure calls.

Save it first:

sw $ra,0($sp)

❌ Forget to restore the stack pointer.

Every allocation:

addi $sp,$sp,-4

must be matched with:

addi $sp,$sp,4

🎤 Lecture Summary

Today, we learned how procedures (functions) work in MIPS Assembly Language.

We studied the MIPS calling convention, used `jal` and `jr` to call and return from procedures, passed arguments through `$a0–$a3`, returned values in `$v0`, and introduced the stack along with the stack pointer `$sp`. We also learned why saving and restoring registers is essential when writing modular Assembly programs.

In the next lecture, we'll explore **Recursion in MIPS Assembly Language**. We'll understand how recursive function calls use the stack, learn to save return addresses and parameters correctly, and implement classic recursive programs such as factorial and Fibonacci.

Thank you for watching, and I'll see you in the next lecture.